

Snowflake Executor Role — Developer Cheatsheet

The role Snowflake uses to check privileges at runtime: not who calls the object, but whose grants apply when it executes.

[!IMPORTANT] Community cheatsheet. Not official Snowflake documentation. For the authoritative reference, see [Snowflake Security docs](#).

Table of Contents

- [The Executor Role Model](#)
- [Start Here](#)
- [Per-Object Quick Reference](#)
- [Stored Procedures](#)
- [Tasks](#)
- [Dynamic Tables](#)
- [UDFs and UDTFs](#)
- [Alerts](#)
- [Streamlit Apps](#)
- [Masking and Row Access Policies](#)
- [Decision Matrix](#)
- [Tips and Gotchas](#)
- [Troubleshooting](#)
- [References](#)

The Executor Role Model

When Snowflake runs a stored procedure, task, UDF, or dynamic table, it evaluates privileges against a specific role. That role is the **executor role**: whose grants are checked, not necessarily the role of the person who triggered execution.

Two context functions expose this at query time:

Function	Returns
<code>CURRENT_ROLE()</code>	The session role of the user running the query
<code>INVOKER_ROLE()</code>	

Function	Returns
	The role Snowflake uses for the currently executing object

These return the same value at the top-level session. They diverge inside objects.

-- At session level: both functions return the same role

```
SELECT CURRENT_ROLE(), INVOKER_ROLE();
```

-- Inside an owner's rights procedure: they diverge

-- CURRENT_ROLE() = the caller's session role

-- INVOKER_ROLE() = the procedure owner's role

Start Here

If you have...	Go to
A stored proc failing with insufficient privileges	Stored Procedures + Decision Matrix
A task that won't resume or stays suspended	Tasks
A dynamic table refresh failing after ownership transfer	Dynamic Tables
A masking policy returning the wrong role	Masking and Row Access Policies
An error message you don't recognize	Troubleshooting

Per-Object Quick Reference

Object	Default Executor	Configurable?	Syntax	Extra Privilege Required
Stored Procedure	Owner role	Yes	EXECUTE AS OWNER \ CALLER \ RESTRICTED CALLER	USAGE on proc
Task	Owner role	No	—	EXECUTE TASK (account- level)
Dynamic Table	Owner role as	Yes (user)	EXECUTE AS USER <name>	IMPERSONATE on target user

Object	Default Executor	Configurable?	Syntax	Extra Privilege Required
	SYSTEM user			
UDF / UDTF	Owner role	No	—	USAGE on function
Alert	Owner role	No	—	EXECUTE ALERT (account-level)
Streamlit App	Owner role	No	—	Owner must have data privileges
Masking / Row Policy	Context-dependent	Via policy body	INVOKER_ROLE() or CURRENT_ROLE()	Enterprise Edition

[!NOTE] Dynamic tables run as an internal SYSTEM user by default. The owner role's grants apply, but `CURRENT_USER()` returns a system identity, not a named user. Use `EXECUTE AS USER` when policies or audit trails require a real user.

Stored Procedures

The `EXECUTE AS` clause sets the execution context at creation time. Default is owner's rights.

Owner's rights: runs with the procedure owner's privileges:

```
CREATE OR REPLACE PROCEDURE myschema.clean_old_orders()
  RETURNS VARCHAR
  LANGUAGE SQL
  EXECUTE AS OWNER
AS
BEGIN
  DELETE FROM orders WHERE created_at < DATEADD(year, -7,
    CURRENT_DATE());
  RETURN 'done';
END;
```

Caller's rights: runs with the caller's current role:

```
CREATE OR REPLACE PROCEDURE myschema.get_my_orders(region VARCHAR)
  RETURNS TABLE (order_id NUMBER, status VARCHAR)
  LANGUAGE SQL
  EXECUTE AS CALLER
AS
```

```

BEGIN
  RETURN TABLE(SELECT order_id, status FROM orders WHERE o_region
    = :region);
END;

```

Restricted caller's rights (Native Apps only):

```

CREATE OR REPLACE PROCEDURE CORE.PROCESS_CONSUMER_DATA()
  RETURNS STRING
  LANGUAGE SQL
  EXECUTE AS RESTRICTED CALLER
AS
BEGIN
  RETURN 'processed';
END;

```

Grant a caller grant to a Native App (run in consumer account):

```
GRANT CALLER USAGE ON DATABASE consumer_db TO APPLICATION my_app;
```

Change execution rights on an existing procedure:

```
ALTER PROCEDURE myschema.clean_old_orders() EXECUTE AS CALLER;
```

Key behavioral differences:

Behavior	Owner's Rights	Caller's Rights
Privileges used	Owner role	Caller's current role
Session variables	Cannot read or set	Can read and set
Database/schema context	Proc's own db/schema	Caller's current db/schema
Source code visibility	Hidden from non-owners	Visible to callers
Nested procs	Entire chain runs as owner's rights	Only if full chain is caller's rights

[!IMPORTANT] Once an owner's rights procedure is anywhere in a nested call chain, every procedure called from it also runs as owner's rights, even those individually set to EXECUTE AS CALLER.

Tasks

Tasks always run as the task owner role. EXECUTE TASK is account-level and must be granted separately. OWNERSHIP alone does not allow a task to run.

Minimum privilege set for a role to own and execute a task:

```
GRANT EXECUTE TASK ON ACCOUNT TO ROLE task_runner_role;  
GRANT OWNERSHIP ON TASK mydb.myschema.my_task TO ROLE  
    task_runner_role;  
GRANT USAGE ON WAREHOUSE task_wh TO ROLE task_runner_role;
```

Resume a task (requires both OWNERSHIP and EXECUTE TASK):

```
ALTER TASK mydb.myschema.my_task RESUME;
```

[!TIP] EXECUTE TASK is granted at the account level, not the object level. Creating a task does not imply it. A role can own a task and still fail to resume it.

Dynamic Tables

Background refreshes run as the owner role. The owner role must retain SELECT on all source objects and USAGE on the warehouse at all times, not just at creation.

Create a dynamic table (owner role needs SELECT on sources + USAGE on warehouse):

```
CREATE OR REPLACE DYNAMIC TABLE mydb.myschema.dt_orders  
    TARGET_LAG = '10 minutes'  
    WAREHOUSE = transform_wh  
AS  
    SELECT order_id, customer_key, total_price  
    FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
```

Use EXECUTE AS USER when masking policies or audit attribution require a named user:

```
GRANT IMPERSONATE ON USER svc_transform TO ROLE transform_role;  
GRANT ROLE transform_role TO USER svc_transform;
```

```
CREATE OR REPLACE DYNAMIC TABLE mydb.myschema.dt_orders  
    TARGET_LAG = '10 minutes'  
    WAREHOUSE = transform_wh  
    EXECUTE AS USER svc_transform  
AS  
    SELECT order_id, customer_key, total_price  
    FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
```

Activate secondary roles during refresh:

```
CREATE OR REPLACE DYNAMIC TABLE mydb.myschema.dt_orders  
    TARGET_LAG = '10 minutes'  
    WAREHOUSE = transform_wh  
    EXECUTE AS USER svc_transform  
    USE SECONDARY ROLES ALL
```

```
AS
SELECT ...;
```

Revert to the default SYSTEM user:

```
ALTER DYNAMIC TABLE mydb.myschema.dt_orders UNSET EXECUTE AS USER;
```

[!IMPORTANT] If the owner role loses SELECT on any source table or USAGE on the warehouse after creation, scheduled refreshes fail and the dynamic table may auto-suspend. Transfer ownership only after verifying the new owner role holds all required privileges.

UDFs and UDTFs

UDFs always execute in the owner's context. This is not configurable.

Grant a role the right to call a UDF:

```
GRANT USAGE ON FUNCTION mydb.myschema.mask_ssn(VARCHAR) TO ROLE
analyst_role;
```

[!NOTE] Since BCR 2023_03: handler code that reads files from a stage executes in the owner's context. Callers must pass file locations via BUILD_SCOPED_FILE_URL(). User stages are not accessible from owner-context UDFs or stored procedures.

Alerts

Alerts run as the alert owner role. Like tasks, OWNERSHIP alone does not allow an alert to fire.

Minimum privilege set:

```
GRANT EXECUTE ALERT ON ACCOUNT TO ROLE alert_owner_role;
GRANT OWNERSHIP ON ALERT mydb.myschema.my_alert TO ROLE
alert_owner_role;
GRANT USAGE ON WAREHOUSE monitor_wh TO ROLE alert_owner_role;
```

Streamlit Apps

Streamlit apps run with owner's rights. The app owner role must hold privileges on any Snowflake objects the app queries, not the viewer's role.

Grant data access to the Streamlit owner role:

```
GRANT USAGE ON DATABASE analytics_db TO ROLE streamlit_owner_role;
GRANT USAGE ON SCHEMA analytics_db.public TO ROLE
    streamlit_owner_role;
GRANT SELECT ON TABLE analytics_db.public.orders TO ROLE
    streamlit_owner_role;
```

[!TIP] A user with VIEWER on your app can see any data the owner role can access, even if the viewer has no direct grants on the underlying tables. Scope the owner role's grants to exactly what the app needs.

Masking and Row Access Policies

Policy body functions evaluate differently depending on the executing object:

Context	INVOKER_ROLE() returns	CURRENT_ROLE() returns
Direct table query	Session role	Session role
View query	View owner role	Session role
Owner's rights stored proc	Proc owner role	Session role
Caller's rights stored proc	Session role	Session role
Task	Task owner role	Session role
UDF	UDF owner role	Session role

Use INVOKER_ROLE() when the policy enforces based on the executing object's owner:

```
CREATE OR REPLACE MASKING POLICY mask_total AS
(val NUMBER) RETURNS NUMBER ->
CASE
    WHEN INVOKER_ROLE() IN ('FINANCE_ROLE') THEN val
    ELSE -1
END;
```

Use IS_GRANTED_TO_INVOKER_ROLE() to check role hierarchy (not just exact match):

```
CREATE OR REPLACE MASKING POLICY mask_ssn AS
(val VARCHAR) RETURNS VARCHAR ->
CASE
    WHEN IS_GRANTED_TO_INVOKER_ROLE('PAYROLL') THEN val
    WHEN IS_GRANTED_TO_INVOKER_ROLE('ANALYST') THEN
        REGEXP_REPLACE(val, '[0-9]', '*', 7)
    ELSE '*****'
END;
```

Use CURRENT_ROLE() when the policy enforces based on the session role:

```
CREATE OR REPLACE MASKING POLICY mask_by_session AS
(val VARCHAR) RETURNS VARCHAR ->
CASE
  WHEN CURRENT_ROLE() IN ('ANALYST') THEN val
  ELSE '*****'
END;
```

Decision Matrix

Goal	Use	Notes
Let a low-privilege role call a proc doing privileged work	EXECUTE AS OWNER	Caller needs only USAGE on the proc; owner's grants do the rest
Proc must read the caller's session variables or current db	EXECUTE AS CALLER	Owner mode is isolated from caller session state
Native App proc needs consumer account object access	EXECUTE AS RESTRICTED CALLER	Consumer uses GRANT CALLER to explicitly authorize access
Dynamic table refresh needs CURRENT_USER() in policy	EXECUTE AS USER <name>	Default SYSTEM user fails policy conditions that check user identity
Masking policy based on who runs the query	CURRENT_ROLE() in policy	Evaluates session role regardless of object ownership
Masking policy based on who owns the executing object	INVOKER_ROLE() in policy	Evaluates view owner, proc owner, task owner, not session role

Tips and Gotchas

- **Ownership transfer on dynamic tables requires a privilege pre-flight.** Grant the new owner role SELECT on all source tables and USAGE on the warehouse *before* transferring ownership. Transferring first causes the next scheduled refresh to fail and the table to auto-suspend.
- **EXECUTE TASK and EXECUTE ALERT are account-level.** Neither is implied by OWNERSHIP. A role can own a task or alert and

still be blocked from running it. Grant both EXECUTE TASK ON ACCOUNT and EXECUTE ALERT ON ACCOUNT explicitly.

- **INVOKER_ROLE() on a view returns the view owner, not the session role.** If your masking policy uses INVOKER_ROLE() and is applied to a table accessed via a view, the function evaluates the view owner role. Use CURRENT_ROLE() if the intent is to enforce based on who is running the query.
- **Owner's rights procs hide source code from callers.** GET_DDL() on an owner's rights proc returns nothing for non-owners. This is intentional. Use it for IP protection.
- **Caller's rights requires the entire call chain.** A single owner's rights proc anywhere in a nested call hierarchy converts all downstream procs to owner's rights, regardless of their individual EXECUTE AS settings.
- **IDENTIFIER() only accepts simple session variable references, not expressions.** IDENTIFIER(\$var || '_suffix') fails everywhere in DDL/DML. Pre-declare all composed names: SET R_NAME = \$PREFIX || '_suffix'; GRANT ... TO ROLE IDENTIFIER(\$R_NAME);
- **USE ROLE IDENTIFIER(...) is not valid syntax.** Use EXECUTE IMMEDIATE: SET r = 'USE ROLE ' || \$ROLE_VAR; EXECUTE IMMEDIATE \$r;
- **GRANT IMPORTED PRIVILEGES ... TO ROLE IDENTIFIER(...) requires EXECUTE IMMEDIATE.** IDENTIFIER() is not supported in this position. Use SET g = '...'; EXECUTE IMMEDIATE \$g;
- **Masking policy argument type must match the column's declared type exactly.** A NUMBER(12,2) column requires (val NUMBER) RETURNS NUMBER. Using FLOAT raises a compile error even when the policy logic is correct.
- **CREATE OR REPLACE MASKING POLICY fails while the policy is attached to a column.** Sequence:
ALTER TABLE t MODIFY COLUMN c UNSET MASKING POLICY; →
CREATE OR REPLACE → re-apply.
- **Session variables don't exist in Dynamic Table refresh context.** EXECUTE AS USER IDENTIFIER(\$var) and FROM IDENTIFIER(\$var) both fail at refresh time. Use a Snowflake Scripting block to embed literal names at DT creation.
- **EXECUTE IMMEDIATE \$g stores command text verbatim as the DT body — not the evaluated SQL.** Use
EXECUTE IMMEDIATE \$\$ DECLARE ... BEGIN sql := '...'; EXECUTE IMMEDIATE sql; END; \$\$;
- **Session variables used in CREATE PROCEDURE DECLARE blocks must exist at compile time.** DECLARE seg VARCHAR DEFAULT \$SEGMENT_FILTER; fails if \$SEGMENT_FILTER has not been SET before the CREATE PROCEDURE statement.
- **RETURNS TABLE column types must match the SELECT output exactly.** Add explicit ::NUMBER, ::FLOAT, ::VARCHAR casts to avoid type mismatch at call time.
- **EXECUTE TASK controls two distinct things.** OWNERSHIP/OPERATE on a task allows ALTER TASK RESUME/SUSPEND (state change only). EXECUTE TASK ON ACCOUNT is required for the scheduler to

actually fire the task body. A task can show state started and silently never execute without the account-level privilege.

- **DESCRIBE PROCEDURE proc_name(arg_type) reveals the execute as property.** information_schema.procedures (SQL standard view) has no execute_as column. Use DESCRIBE PROCEDURE or SHOW PROCEDURES + DESCRIBE PROCEDURE to confirm execution mode.

Troubleshooting

Error or Symptom	Cause	Fix
Cannot execute task, EXECUTE TASK privilege must be granted to owner role	EXECUTE TASK is account-level; OWNERSHIP does not imply it	GRANT EXECUTE TASK ON ACCOUNT TO ROLE <owner_role>
Dynamic table refreshes fail after GRANT OWNERSHIP	New owner role lacks SELECT on sources or USAGE on warehouse	Grant privileges to new role before transfer; ALTER DYNAMIC TABLE ... RESUME
User stage file access is not allowed within an owner's rights SP or UDF	Owner-context objects cannot access user stages	Use named internal stages; pass paths via BUILD_SCOPED_FILE_URL()
Masking policy unmask data through a view despite unprivileged session role	INVOKER_ROLE() evaluates the view owner, not the session role	Switch to CURRENT_ROLE() for session-role-based enforcement
Owner's rights proc fails: SQL variable ... is not defined	Owner mode cannot read caller's session variables	Pass the variable as an explicit proc parameter
Nested proc unexpectedly runs as owner's rights	Entire chain inherits owner's rights once any proc in it is owner's rights	Extract logic into standalone procs; restructure so owner's rights proc does not call caller's rights procs
Insufficient privileges to execute ALERT IDENTIFIER(\$var \\ 'suffix')	EXECUTE ALERT is account-level; OWNERSHIP does not imply it	GRANT EXECUTE ALERT ON ACCOUNT TO ROLE <owner_role>

Error or Symptom	Cause	Fix
raises invalid identifier	IDENTIFIER() requires a simple session variable reference, not an expression	Pre-declare: SET R = \$PREFIX \ \ '_suffix'; then IDENTIFIER(\$R)
USE ROLE IDENTIFIER(...) raises SQL syntax error	USE ROLE does not support IDENTIFIER()	SET r = 'USE ROLE ' \ \ \$ROLE; EXECUTE IMMEDIATE \$r;
CREATE OR REPLACE MASKING POLICY fails on an attached policy	Policy is referenced by a column — cannot replace while attached	ALTER TABLE t MODIFY COLUMN c UNSET MASKING POLICY; then replace, then re-apply
Masking policy raises type mismatch error despite correct logic	Policy argument type doesn't match column data type exactly (e.g. FLOAT vs NUMBER(12,2))	Match exactly: NUMBER(12,2) column → (val NUMBER) RETURNS NUMBER
Task state is started but body never executes	OWNERSHIP/OPERATE allows ALTER TASK RESUME (state change) but not scheduler execution	GRANT EXECUTE TASK ON ACCOUNT TO ROLE <owner_role>
SQL variable ... is not defined inside CREATE PROCEDURE body	Session variable missing at compile time	SET my_var = 'value'; before the CREATE PROCEDURE statement
Dynamic table body contains EXECUTE IMMEDIATE \$g literally	Snowflake stores the DT definition verbatim; session variable is not expanded	Use scripting block: EXECUTE IMMEDIATE \$\$ DECLARE ... BEGIN sql := '...'; EXECUTE IMMEDIATE sql; END; \$\$;
DESCRIBE PROCEDURE needed to check execution mode	information_schema.procedures has no execute_as column	DESCRIBE PROCEDURE proc_name(arg_type); returns the execute as property

References

- [Caller's Rights and Owner's Rights Stored Procedures](#)
- [Restricted Caller's Rights](#)
- [Dynamic Table Access Control](#)
- [Dynamic Tables: EXECUTE AS USER](#)
- [INVOKER_ROLE Function](#)
- [IS_GRANTED_TO_INVOKER_ROLE Function](#)
- [Advanced Column-Level Security Topics](#)

- Task Access Control